

[M-01] Settlement Casting Chain Can Cause DOS or Pool Accounting Corruption

Summary

The `exercise()` and `_computeExchange()` functions in `CollateralTracker.sol` use a dangerous casting chain: `int256` $\rightarrow$ `uint256` $\rightarrow$ `uint128`. When the intermediate `int256` value becomes negative (which can happen during large position exercises), the `uint256` cast produces an astronomically large value, causing `toUint128()` to either revert (DOS) or truncate to a garbage value (accounting corruption).

Vulnerability Detail

The Vulnerable Pattern

```
// CollateralTracker.sol:1031-1032
int256 updatedAssets = int256(uint256(s_poolAssets)) - swappedAmount
s_poolAssets = uint256(updatedAssets).toUint128();
//          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//          DANGEROUS CASTING CHAIN

// CollateralTracker.sol:1079-1080
int256 updatedAssets = int256(uint256(s_poolAssets)) + swappedAmount
s_poolAssets = uint256(updatedAssets).toUint128();
```

Step-by-Step Breakdown

```
// Given:
uint128 s_poolAssets = 100 ether;
int128 swappedAmount = 150 ether; // Larger than pool assets

// Step 1: Convert to int256
int256 step1 = int256(uint256(s_poolAssets));
// step1 = 100 ether ✓

// Step 2: Subtract swappedAmount
int256 updatedAssets = step1 - swappedAmount;
// updatedAssets = 100 - 150 = -50 ether
// updatedAssets = -5000000000000000000000000000000000000000000000000000000 (negative!)

// Step 3: Cast to uint256
uint256 asUInt = uint256(updatedAssets);
// asUInt = 11579208923731619542357098500868790785326998466564056403
// ( $2^{256} - 50$  ether due to two's complement)

// Step 4: toUint128()
// asUInt > type(uint128).max
// RESULT: Either REVERT (SafeCast) or TRUNCATE (unchecked cast)
```

When This Occurs

The condition `swappedAmount > s_poolAssets` can happen when:

1. **Large Position Exercise:** User exercises a position with notional value exceeding pool's available assets
2. **Low Liquidity Pool:** Pool has few deposits relative to outstanding positions
3. **ITM Position Settlement:** In-the-money positions require larger swaps

Impact

Scenario A: DOS (If using SafeCast)

1. User tries to exercise position
2. `swappedAmount > s_poolAssets`
3. `toUInt128()` detects overflow → reverts
4. User cannot exercise their position
5. Position locked until pool gets more deposits

Scenario B: Accounting Corruption (If unchecked cast)

1. User exercises position
2. Negative intermediate value
3. Truncation produces garbage `s_poolAssets` value
4. Pool accounting becomes wildly incorrect
5. Subsequent operations use corrupted state
6. Potential for fund extraction or permanent lock

Likelihood Assessment

FACTOR	VALUE
Requires large position	Medium frequency
Requires low pool liquidity	Medium frequency
Combined probability	Low-Medium
Impact if triggered	High (DOS) or Critical (corruption)

Proof of Concept

```
function test_M01_SettlementCastingChain() public {
    uint128 s_poolAssets = 100 ether;
    int128 swappedAmount = 150 ether; // Larger than pool assets

    // Step 1
    int256 step1 = int256(uint256(s_poolAssets));
    console.log("Step 1:", uint256(step1)); // 100 ether

    // Step 2
    int256 updatedAssets = step1 - swappedAmount;
    console.logInt(updatedAssets); // -50 ether

    // Step 3
    uint256 asUint = uint256(updatedAssets);
    console.log("Step 3:", asUint);
    // 1157920892373161954235709850086879078532699846656405640394075

    // Step 4 would revert or truncate
    assertTrue(asUint > type(uint128).max, "Exceeds uint128");
}
```

Test Output

```
≡ M-01: Settlement Casting Chain ≡
```

Initial s_poolAssets: 100 ETH
swappedAmount: 150 ETH

Step 1: int256(uint256(s_poolAssets)): 100
Step 2: updatedAssets = step1 - swappedAmount
-50000000000000000000000000

!!! updatedAssets is NEGATIVE !!!
Step 3: uint256(updatedAssets) =
115792089237316195423570985008687907853269984665640564039407584007913

Step 4: toUint128() would either:

- REVERT (if using SafeCast)
- TRUNCATE to garbage value

Result: Pool accounting corrupted or TX reverts

Run the PoC

```
FOUNDRY_PROFILE=poc forge test --match-contract M01_M04_Arithmetic_V
```

Tools Used

- Manual code review
- Foundry for PoC development

Recommended Mitigation

Option 1: Pre-Validate Amounts (Recommended)

```
function _computeExchange(...) internal {
    // Check before dangerous arithmetic
    if (swappedAmount > int256(uint256(s_poolAssets))) {
        revert InsufficientPoolAssets();
    }

    int256 updatedAssets = int256(uint256(s_poolAssets)) - swappedAm
    // Now guaranteed non-negative
    s_poolAssets = uint128(uint256(updatedAssets));
}
```

Option 2: Use Signed Arithmetic Throughout

```
// Store as int128 to properly handle negative values
int128 internal s_poolAssets;

function _computeExchange(...) internal {
    s_poolAssets = s_poolAssets - swappedAmount;

    // Validate result is reasonable
    require(s_poolAssets ≥ 0, "Pool assets negative");
}
```

Option 3: Add Explicit Bounds Check

```
int256 updatedAssets = int256(uint256(s_poolAssets)) - swappedAmount

// Explicit check before cast
if (updatedAssets < 0) {
    revert NegativePoolAssets();
}
if (updatedAssets > type(uint128).max) {
    revert PoolAssetsOverflow();
}

s_poolAssets = uint128(uint256(updatedAssets));
```